Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

Доповідь з дисципліни
«Об'єктно-орієнтоване програмування»

на тему:
«Розробка графічного редактора об’єктів»

(друга лабороторна робота)

Виконав:

Студент групи ІМ-51
Круковський Володимир Шон Ігорович
номер у списку групи: 20

Київ – 2026

Мета роботи:

Отримати вміння та навички використання інкапсуляції, абстракції типів, успадкування та поліморфізму на основі класів, запрограмувавши простий графічний редактор в об'єктно-орієнтованому стилі.

Завдання згідно варіанту:

Згідно з індивідуальним номером у журналі ($Ж = 20$), параметри завдання є наступними:

1. Масив вказівників для об'єктів `Shape`: оскільки $20 \bmod 3 = 2 \neq 0$, використовується статичний масив `Shape *pcshape[120]`, де загальна кількість елементів $N = 20 + 100 = 120$.
2. «Гумовий» слід (Rubberband) при вводі: оскільки $20 \bmod 4 = 0$, це суцільна лінія чорного кольору.
3. Прямокутник:

- Ввід: $20 \bmod 2 = 0 \rightarrow$ по двом протилежним кутам.
- Відображення: $20 \bmod 5 = 0 \rightarrow$ чорний контур з білим заповненням.

4. Еліпс:

- Ввід: $20 \bmod 2 = 0 \rightarrow$ від центру до одного з кутів охоплюючого прямокутника.
- Відображення: $20 \bmod 5 = 0 \rightarrow$ чорний контур еліпсу без заповнення.

5. Позначка поточного типу об'єкта: оскільки $20 \bmod 2 = 0 \rightarrow$ в меню (метод `OnInitMenuPOPUP`).

Головний файл: Lab2.cpp:

```
#include <windows.h>
#include "Shape.h"
#define ID_FILE_CLEAR   101
#define ID_FILE_EXIT    102
#define ID_OBJ_POINT    201
#define ID_OBJ_LINE     202
#define ID_OBJ_RECT     203
#define ID_OBJ_ELLIPSE  204

HINSTANCE hInst;
HWND g_hWnd = NULL;

const int MAX_SHAPES = 120;
```

```
Shape* pcshape[MAX_SHAPES] = { nullptr };
int shapeCount = 0;

int currentShapeType = 1;
bool isDrawing = false;
int startX = 0, startY = 0;
int currentX = 0, currentY = 0;

LRESULT CALLBACK WndProc(HWND, UINT, WPARAM, LPARAM);

int APIENTRY wWinMain(_In_ HINSTANCE hInstance, _In_opt_ HINSTANCE hPrevInstance, _In_
LPWSTR lpCmdLine, _In_ int nCmdShow) {
    hInst = hInstance;
    WNDCLASSEXW wcex = { sizeof(WNDCLASSEX) };
    wcex.style = CS_HREDRAW | CS_VREDRAW;
    wcex.lpfnWndProc = WndProc;
    wcex.hInstance = hInstance;
    wcex.hCursor = LoadCursor(nullptr, IDC_ARROW);
    wcex.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);
    wcex.lpszClassName = L"Lab2Class";
    RegisterClassExW(&wcex);

    HMENU hMenu = CreateMenu();
    HMENU hFileMenu = CreateMenu();
    HMENU hObjMenu = CreateMenu();
    HMENU hHelpMenu = CreateMenu();

    AppendMenuW(hFileMenu, MF_STRING, ID_FILE_CLEAR, L"Clear");
    AppendMenuW(hFileMenu, MF_SEPARATOR, 0, NULL);
    AppendMenuW(hFileMenu, MF_STRING, ID_FILE_EXIT, L"Exit");

    AppendMenuW(hObjMenu, MF_STRING, ID_OBJ_POINT, L"Point");
    AppendMenuW(hObjMenu, MF_STRING, ID_OBJ_LINE, L"Line");
    AppendMenuW(hObjMenu, MF_STRING, ID_OBJ_RECT, L"Rectangle");
    AppendMenuW(hObjMenu, MF_STRING, ID_OBJ_ELLIPSE, L"Ellipse");

    AppendMenuW(hMenu, MF_POPUP, (UINT_PTR)hFileMenu, L"File");
    AppendMenuW(hMenu, MF_POPUP, (UINT_PTR)hObjMenu, L"Objects");
    AppendMenuW(hMenu, MF_POPUP, (UINT_PTR)hHelpMenu, L"Help");

    g_hWnd = CreateWindowW(L"Lab2Class", L"Lab 2 – Variant 20",
        WS_OVERLAPPEDWINDOW, CW_USEDEFAULT, 0, 800, 600, nullptr, hMenu, hInstance,
nullptr);

    if (!g_hWnd) return FALSE;
    ShowWindow(g_hWnd, nCmdShow);
    UpdateWindow(g_hWnd);

    MSG msg;
    while (GetMessage(&msg, nullptr, 0, 0)) {
        TranslateMessage(&msg);
        DispatchMessage(&msg);
    }

    for (int i = 0; i < shapeCount; i++) {
        delete pcshape[i];
    }

    return (int)msg.wParam;
}
```

```
LRESULT CALLBACK WndProc(HWND hWnd, UINT message, WPARAM wParam, LPARAM lParam) {
    switch (message) {
    case WM_INITMENUPOPUP: {
        HMENU hMenu = (HMENU)wParam;
        CheckMenuItem(hMenu, ID_OBJ_POINT, MF_BYCOMMAND | (currentShapeType == 1 ?
MF_CHECKED : MF_UNCHECKED));
        CheckMenuItem(hMenu, ID_OBJ_LINE, MF_BYCOMMAND | (currentShapeType == 2 ?
MF_CHECKED : MF_UNCHECKED));
        CheckMenuItem(hMenu, ID_OBJ_RECT, MF_BYCOMMAND | (currentShapeType == 3 ?
MF_CHECKED : MF_UNCHECKED));
        CheckMenuItem(hMenu, ID_OBJ_ELLIPSE, MF_BYCOMMAND | (currentShapeType == 4 ?
MF_CHECKED : MF_UNCHECKED));
        break;
    }
    case WM_COMMAND: {
        int id = LOWORD(wParam);
        if (id == ID_FILE_CLEAR) {
            for (int i = 0; i < shapeCount; i++) {
                delete pcshape[i];
                pcshape[i] = nullptr;
            }
            shapeCount = 0;
            InvalidateRect(hWnd, NULL, TRUE);
        }
        else if (id == ID_FILE_EXIT) {
            DestroyWindow(hWnd);
        }
        else if (id >= ID_OBJ_POINT && id <= ID_OBJ_ELLIPSE) {
            currentShapeType = id - ID_OBJ_POINT + 1;
        }
        break;
    }
    case WM_LBUTTONDOWN: {
        isDrawing = true;
        startX = LOWORD(lParam);
        startY = HIWORD(lParam);
        currentX = startX;
        currentY = startY;
        SetCapture(hWnd);
        break;
    }
    case WM_MOUSEMOVE: {
        if (isDrawing) {
            HDC hdc = GetDC(hWnd);
            SetROP2(hdc, R2_NOTXORPEN);

            HPEN hPenOld = (HPEN)SelectObject(hdc, CreatePen(PS_SOLID, 1, RGB(0, 0,
0)));

            if (currentShapeType == 2) {
                MoveToEx(hdc, startX, startY, NULL); LineTo(hdc, currentX, currentY);
                MoveToEx(hdc, startX, startY, NULL); LineTo(hdc, LOWORD(lParam),
HIWORD(lParam));
            }
            else if (currentShapeType == 3) {
                Rectangle(hdc, startX, startY, currentX, currentY);
                Rectangle(hdc, startX, startY, LOWORD(lParam), HIWORD(lParam));
            }
            else if (currentShapeType == 4) {
                Ellipse(hdc, startX, startY, currentX, currentY);
                Ellipse(hdc, startX, startY, LOWORD(lParam), HIWORD(lParam));
```

```
            }

            currentX = LOWORD(lParam);
            currentY = HIWORD(lParam);

            DeleteObject(SelectObject(hdc, hPenOld));
            ReleaseDC(hWnd, hdc);
        }
        break;
    }
    case WM_LBUTTONUP: {
        if (isDrawing) {
            isDrawing = false;
            ReleaseCapture();

            int endX = LOWORD(lParam);
            int endY = HIWORD(lParam);

            if (shapeCount < MAX_SHAPES) {
                if (currentShapeType == 1) {
                    pcshape[shapeCount++] = new PointShape(startX, startY, RGB(0, 0,
0));
                }
                else if (currentShapeType == 2) {
                    pcshape[shapeCount++] = new LineShape(startX, startY, endX, endY,
RGB(0, 0, 0));
                }
                else if (currentShapeType == 3) {
                    pcshape[shapeCount++] = new RectShape(startX, startY, endX, endY,
RGB(0, 0, 0));
                }
                else if (currentShapeType == 4) {
                    pcshape[shapeCount++] = new EllipseShape(startX, startY, endX, endY,
RGB(0, 0, 0));
                }
            }
            InvalidateRect(hWnd, NULL, TRUE);
        }
        break;
    }
    case WM_PAINT: {
        PAINTSTRUCT ps;
        HDC hdc = BeginPaint(hWnd, &ps);

        for (int i = 0; i < shapeCount; i++) {
            if (pcshape[i] != nullptr) {
                pcshape[i]->Draw(hdc);
            }
        }

        EndPaint(hWnd, &ps);
        break;
    }
    case WM_DESTROY:
        PostQuitMessage(0);
        break;
    default:
        return DefWindowProc(hWnd, message, wParam, lParam);
    }
    return 0;
}
```

Shape.h:

```
#pragma once
#include <windows.h>

 class Shape {
protected:
    int startX, startY;
    int endX, endY;
    COLORREF color;
public:
    Shape(int x1, int y1, int x2, int y2, COLORREF col)
        : startX(x1), startY(y1), endX(x2), endY(y2), color(col) {
    }
    virtual ~Shape() {}
     virtual void Draw(HDC hdc) = 0; // Чисто віртуальна функція (поліморфізм)
};

// point
 class PointShape : public Shape {
public:
    PointShape(int x1, int y1, COLORREF col) : Shape(x1, y1, x1, y1, col) {}
    void Draw(HDC hdc) override {
        HPEN hPen = CreatePen(PS_SOLID, 1, color);
        HBRUSH hBrush = CreateSolidBrush(color);
        HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);
        HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, hBrush);

        Ellipse(hdc, startX - 2, startY - 2, startX + 2, startY + 2);

        SelectObject(hdc, hOldPen);
        SelectObject(hdc, hOldBrush);
        DeleteObject(hPen);
        DeleteObject(hBrush);
    }
};

// line
class LineShape : public Shape {
public:
    LineShape(int x1, int y1, int x2, int y2, COLORREF col) : Shape(x1, y1, x2, y2, col)
{}
    void Draw(HDC hdc) override {
        HPEN hPen = CreatePen(PS_SOLID, 1, color);
        HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);
        MoveToEx(hdc, startX, startY, NULL);
        LineTo(hdc, endX, endY);
        SelectObject(hdc, hOldPen);
        DeleteObject(hPen);
    }
};

// rectanlge
class RectShape : public Shape {
public:
    RectShape(int x1, int y1, int x2, int y2, COLORREF col) : Shape(x1, y1, x2, y2, col)
{}
    void Draw(HDC hdc) override {
```

```
        HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0)); // Чорний контур
        HBRUSH hBrush = (HBRUSH)GetStockObject(WHITE_BRUSH); // Біле заповнення
        HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);
        HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, hBrush);

        Rectangle(hdc, startX, startY, endX, endY);

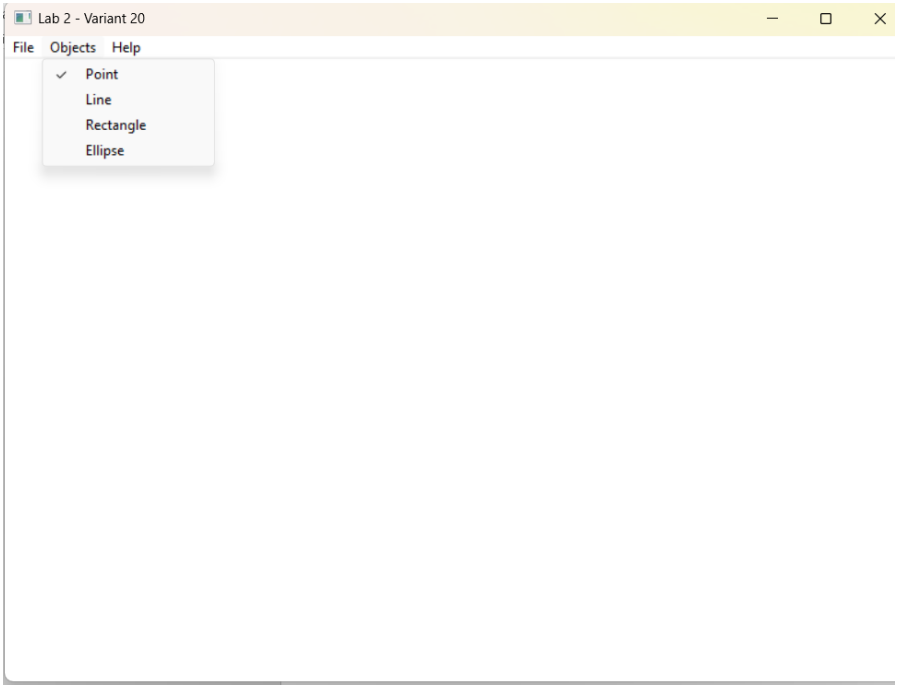
        SelectObject(hdc, hOldPen);
        SelectObject(hdc, hOldBrush);
        DeleteObject(hPen);
    }
};

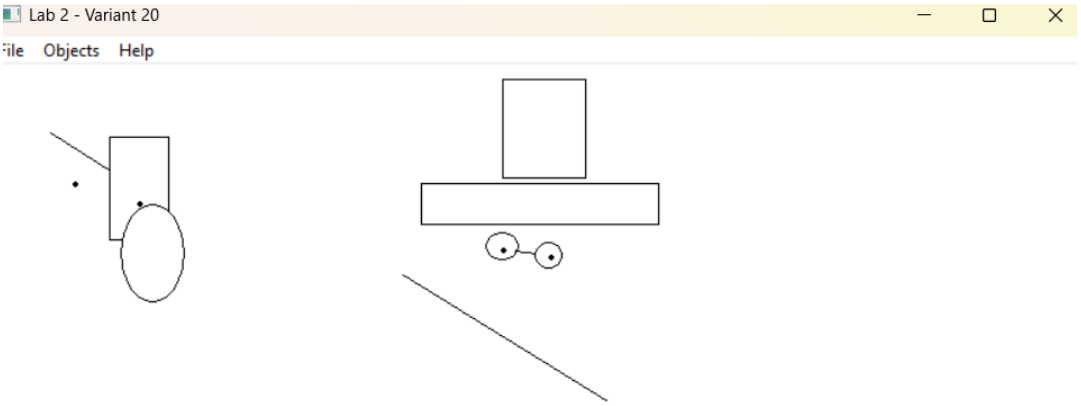
// 4. ellips
class EllipseShape : public Shape {
public:
    EllipseShape(int x1, int y1, int x2, int y2, COLORREF col) : Shape(x1, y1, x2, y2,
col) {}
    void Draw(HDC hdc) override {
        HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0)); // Чорний контур
        HBRUSH hBrush = (HBRUSH)GetStockObject(WHITE_BRUSH); // Білий фон (без
заповнення)
        HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);
        HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, hBrush);

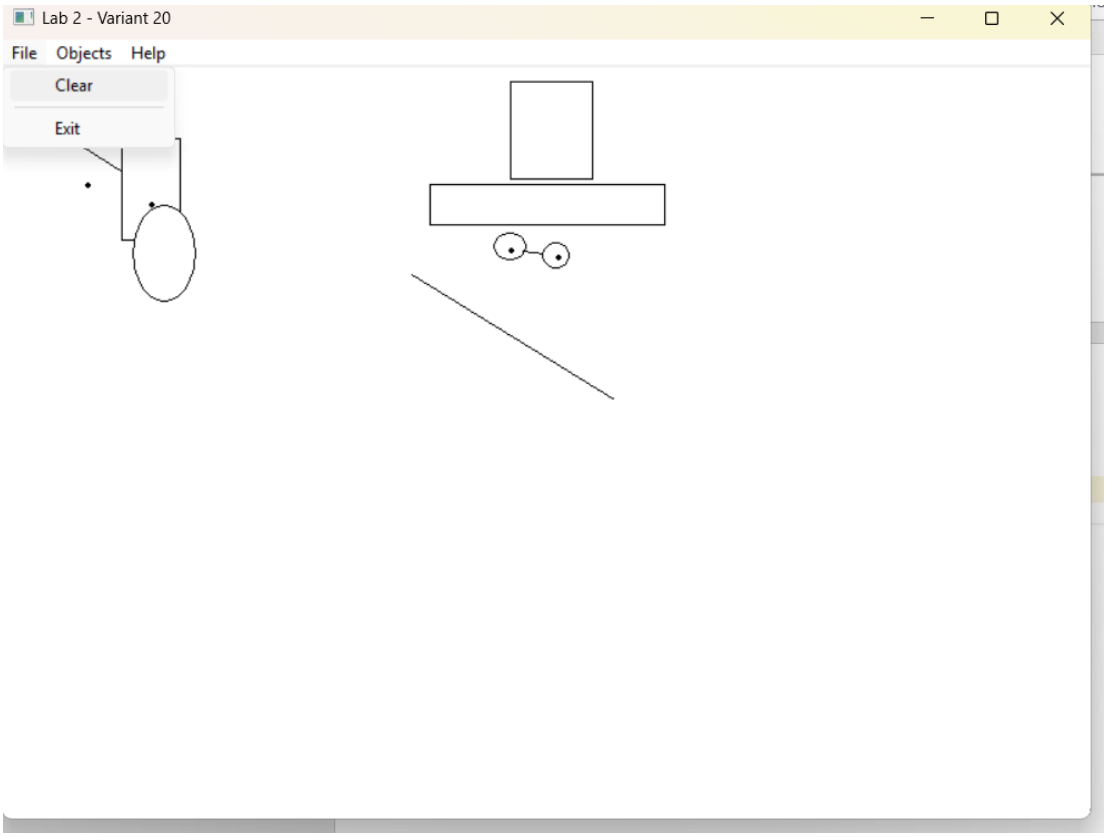
        Ellipse(hdc, startX, startY, endX, endY);

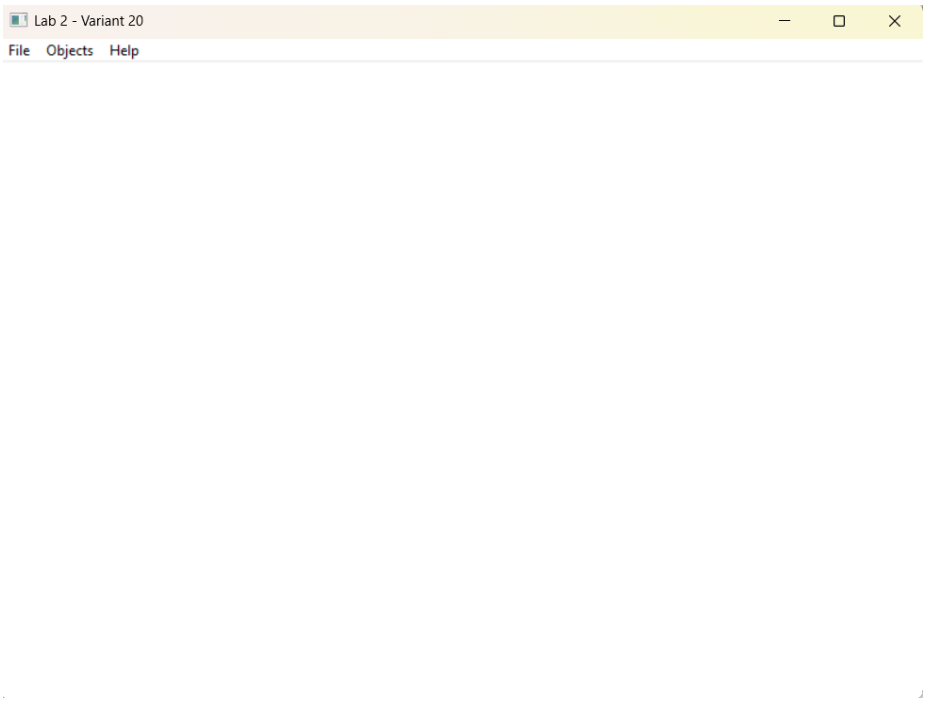
        SelectObject(hdc, hOldPen);
        SelectObject(hdc, hOldBrush);
        DeleteObject(hPen);
    }
};
```

Скріншоти:




Lab 2 - Variant 20
File Objects Help


Lab 2 - Variant 20
File Objects Help
Clear
Exit



Висновки

Під час виконання лабораторної роботи №2 я засвоїв і практично застосував основні принципи об'єктно-орієнтованого програмування: інкапсуляцію, наслідування та поліморфізм на мові C++. За допомогою середовища Visual Studio та бібліотеки Win32 API було створено повноцінний десктопний графічний редактор.

У ході роботи було реалізовано:

- Статичний масив вказівників `Shape* pcshape[120]` для зберігання створених фігур.

- Динамічне введення об'єктів за допомогою миші в режимі «гумового сліду» із застосуванням XOR-режиму малювання (`R2_NOTXORPEN`).

- Механізм поліморфного виведення графічних примітивів усередині обробника `WM_PAINT` за допомогою віртуального методу `Draw()`.

- Динамічне маркування активного інструмента в меню за допомогою повідомлення `WM_INITMENUPOPUP`.